

Chapter 3

The Relevance of Computation

A bird is an instrument working according to mathematical law, which instrument it is within the capacity of man to reproduce with all its movements.

Leonardo da Vinci (1452–1519)

Some Background: Formalism, Symbols, and Mechanisms

The possibility of imitating life by artifact has intrigued people throughout history (Cohen, 1966); but it is only in the second half of this century that the possibility of using the special type of artifact called a computer has been considered seriously as a means of understanding mental phenomena. What is different about this latest interest is that the focus is not primarily on the imitation of movements (as was the case with early clockwork mechanisms) but on the *imitation* (to the extent that this word is even appropriate) of certain unobservable internal processes. This notion became conceivable only with the gradual emergence, in several disparate areas of intellectual development, of a certain abstract way of understanding *mechanism*.

It is difficult to say where the relevant ideas originated, since the structure-function or form-substance distinction implicit in notions of symbol and mechanism dates from the ancient Greeks. In modern times, one of the earliest concerns with mechanism is closely related to attempts to develop a completely formal, content-free foundation for mathematics. The “Hilbert program” is one of the most ambitious attempts to build up mathematics by purely formal means, without regard to questions of what the formalism was about. In the work of Frege, as well as Russell and Whitehead, some of this enterprise was successful. On the other hand, a great intellectual achievement of the modern age has been the demonstration by purely formal means that the ultimate goal of complete formalization is, in principle, not achievable. (The original work was done by Godel and subsequently by Turing, Church, Post, and others; see the collection of papers in Davis, 1965.)

The same work that provided demonstrations of some in-principle limitations of formalization provided demonstrations of formalization's universality as well. Thus Alan Turing, Emil Post, and Alonzo Church independently developed distinct formalisms that were shown to be complete in the sense that they are powerful enough to formally (that is, "mechanistically") generate all sequences of expressions capable of interpretation as proofs, and hence, can generate all provable theorems of logic. In Turing's work this took the form of showing that there exists a universal mechanism (called the Universal Turing Machine [UM]) that can simulate any mechanism describable in its formalism. The machine does this by accepting a description of the mechanism to be simulated; it then carries out a procedure which, in a special sense, is formally equivalent to the procedure that would have been followed by the machine whose description it was given. Here, the sense of formal equivalence is that the inputs and outputs of the UM can be decoded uniformly as the inputs and outputs of the machine being simulated. We say that the UM "computes the same function" as the target machine, where, by *same function*, we mean the same input-output pairs or the same *extension* of the function. That is, there is no implication that the UM performs the same steps as the target machine. That would be a stronger sense of equivalence, one to which we return presently.

What is interesting about Turing's work from our point of view is that, to derive these results concerning the limits and universality of formalization, it was necessary to understand the notions of proof and deduction in a formal system in terms of the manipulation of symbol tokens or marks on a piece of paper, where the manipulation is specified "mechanistically" in a manner entirely independent of the way the symbols might be interpreted. This understanding took a long time to develop, for, in a sense, it meant crystallization of the platonic ideal of "pure form" divorced from all content. Logic became a game played with meaningless symbol tokens according to certain rules of form (that is, syntactic rules).

The only property symbol tokens have in these systems is a nominal one: *type identity*. A particular symbol token has to be recognizable as belonging to a certain type regardless of the context in which it occurs. Another way of stating this is to say that symbol tokens were themselves assumed to be unstructured; one could not say two symbol tokens are *more* or *less* similar, or that they are similar in certain *ways*, any more than one can say two electrons are more or less similar. Further, it was assumed that one could make indistinguishable copies of symbols (in order to identify two marks as distinct occurrences or tokens of the same symbol).

The notion of a discrete atomic symbol is the basis of all formal understanding. Indeed, it is the basis of all systems of thought, expression, or calculation for which a *notation* is available. It is important to stress that such an idea not only has deep roots in what is sometimes called the intellectualist tradition but that no one has succeeded in defining any other type of atom from which formal understanding can be derived. Small wonder, then, that many of us are reluctant to dispense with this foundation in cognitive psychology under frequent exhortations to accept symbols with such varied intrinsic properties as continuous or analogue properties. Unless these notions can be reduced to either atomic symbol foundations or to physical foundations, they remain intellectual orphans, hence, are a poor basis for explanation. The problem is, such notions lack systematic foundations; we do not know what can be done with them. To state the matter more precisely, when we refer to such symbolic (or mental) entities, there is an important sense in which we do not understand what we are talking about! (See chapter 7.)

It was the development of the notion of the universality of formal mechanism, first introduced in the work on foundations of mathematics in the 1930s, that provided the initial impetus for viewing the mind as a symbol-processing system. Allen Newell (1980) provides an insightful way of viewing both the universality and the limitations of formal symbol-manipulation systems. I borrow Newell's points in the next several paragraphs.

Universality implies that a formal symbol-processing mechanism can produce *any* arbitrary input-output function. Now, even within the confines of symbolic functions, where we abstract from the physical properties of inputs and outputs, such arbitrary flexibility seems paradoxical. After all, if the mechanism is deterministic, it obviously can realize only one function—the one that pairs its inputs with its determinate outputs. To view the mechanism as universal, its inputs must be partitioned into distinct components. One component is assigned a privileged interpretation as instructions or as a specification of a particular input-output function, the other is treated as the proper input to that function. Such a partition is essential for defining a universal mechanism. This way of looking at the phenomenon of universality is important also because it shows that there can only be arbitrary plasticity of behavior if there is some *interpretation* of its inputs and outputs (though, of course, the converse does not hold: interpretation is a necessary but not sufficient condition for universality).

Once we take the step of viewing some of the input symbols as designating or identifying a certain function, we encounter a remarkable difficulty: there are simply too many potential input-output functions

to allow specification by a finite set of instructions—or by any other finite means. The set of functions is not only infinite, it is uncountable. That is, it cannot be mapped onto the integers in a one-to-one fashion; therefore, the functions cannot be listed even on an infinitely long list. One of this century's great intellectual achievements has been to show that there exist functions that cannot be specified by finitary means. Thus it seems that, contrary to the claim made above, the input-output behavior of a mechanistic system is not really arbitrarily plastic; if such behavior is to be described finitely, the behavior cannot be arbitrarily varied. On the other hand, it is natural to ask just how flexible such behavior can be. What sort of behaviors *can* be described finitely?

It might understandably be objected that what behaviors can be described finitely surely must depend on how one describes the behaviors. I can describe behavior as, say, "adaptive," thereby describing an infinite set of behaviors in a simple, flexible way. Clearly, we want to exclude such highly general, partial descriptions. For the purpose of providing a theory of the behavior, we need a description that somehow captures the entire class of behaviors without requiring additional knowledge or additional intelligent interpretation. If we could achieve that description we would have a theory that provides a noncircular analysis of the behavior into simpler behaviors, in which case we might even possess a way of capturing the extension of the set of behaviors or of providing a way to decide, for a given input-output pair, whether the pair is an instance of that set of behaviors. But how can we constrain our theoretical apparatus to provide only this kind of exhaustive characterization of behavior? The question has no general answer. What there is, however, is another remarkable empirical result. In the last fifty years a rather large variety of formalisms have been developed, all with the idea of their being maximally expressive yet mechanistic in spirit. Intuitively, all these formalisms can specify functions in a way that does not presuppose an intelligent agent to apply the specification. In every case, however, it turned out that as each new scheme was proposed, it was found to be intertranslatable with all the other schemes. It appears that every scheme picks out exactly the same set of finitely specifiable functions—the "recursive" or "effectively computable functions." Many different schemes can be used to finitely specify functions in this class; thus many different ways are available for capturing the notion of a universal mechanism. It appears, though, that they are all variations on the same underlying idea—that of a completely specifiable set of behaviors or behavioral propensities—consequently, it is generally accepted that they provide a definition of the intuitive notion of a "mechanistic process."

We thus have a notion of maximally plastic behavior that circum-

scribes *interpreted* behavior, just as the notion “nomological,” or lawlike, circumscribes *physically possible* behavior. This extreme plasticity in behavior is one reason why computers have been viewed all along as artifacts possibly capable of exhibiting intelligence. Those not familiar with this view frequently have misunderstood the capacity of computers. For example, the Gestalt psychologist Wolfgang Kohler (1947) viewed machines as too rigid to serve as models of mental activity, claiming that mental activity is governed by “dynamic factors”—an example of which are self-distributing field effects such as those that cause magnetic fields to be redistributed when new pieces of metal are introduced—as opposed to structurally rigid “topographical factors.” “To the degree to which topographical conditions are rigidly given,” Kohler (1947, p. 65) says, “and not to be changed by dynamic factors, their existence means the exclusion of certain forms of function, and the restriction of the processes to the possibilities compatible with those conditions. . . . This extreme relation between dynamic factors and imposed topographical conditions is almost entirely realized in typical machines . . . we do not construct machines in which dynamic factors are the main determinants of the form of operation.” That computers violate this claim is one of their unique and most important characteristics. The topographic structure of computers is completely rigid, yet they are capable of maximal plasticity of function. It is this property that led Turing (1950) to speculate that computers are, in principle, capable of exhibiting intelligent behavior. For example, he devoted an important early philosophical paper (Turing [1950]) to an exposition of this idea, arguing that a computer can, in principle, be made to exhibit intelligent activity to an arbitrary degree. Turing argued that a machine would qualify as intelligent if it could play the “imitation game” successfully, that is, fool a human observer with whom it could communicate only through a teletype so that the observer could not discriminate between it and another person. The possibility of a computer successfully passing what has become known as the Turing test is based entirely on recognition of the plasticity of behavior entailed by symbolic systems, which can be programmed to behave according to any finitely specifiable function.

It should be recognized that plasticity is only part of the story. Even if it were true that a Turing machine can behave in a manner indistinguishable from that of a human, given certain constraints on what is allowed to count as appropriate observations of behavior, producing such behavioral mimicry is a far different matter from providing an *explanation* of that behavior. Explaining behavior is a much more stringent task than generating behavior. Specifically, explanation entails capturing the underlying generalizations as perspicuously as possible

and relating them to certain universal principles. To do that, I have argued, it is necessary to appeal to, among other things, semantically interpreted representations. Thus I provided some indications that explanatory adequacy requires a much finer-grain correspondence between a computational model and an organism than is implied by input-output, or “Turing” equivalence. Most of the remainder of this book is devoted to examining the constraints a computational system must satisfy in order for it to serve as an explanation. For the time being, however, I am concerned with showing that the notion of computational mechanism and formal symbol system implicit in the preceding discussion is in fact closer in spirit to the requirements of cognitive explanation than is apparent from the arguments based on plasticity alone.

We already have one indication that this is the case, from the fact that maximal plasticity is achievable only by introducing interpretation of some of the inputs. It must be the case that some part of the input is viewed as consisting of symbols that designate instructions or that otherwise specify the function in a finite manner. This much semantics is indispensable. If, however, we are, in fact, to view the mechanism as carrying out a certain function on semantically interpreted inputs and outputs—say, on numbers—then the rest of the symbols in the system must also be interpreted, as must the rules for transforming expressions. Further, in this case, the explanation of why the system produces certain interpreted outputs (for example, why it prints out certain *numbers*) must appeal to numerical algorithms, to rules of arithmetic applied to representations of numbers. As we saw in chapter 2, these processes must appeal to semantically interpreted representations. When we take into account the requirements of explanation, we find that a much finer-grained analysis of the symbol-processing functions carried out by the computational mechanisms becomes relevant. In the next section I introduce the notion of computation as the rule-governed transformation of formal expressions viewed as interpreted symbolic codes.

Computation as a Symbolic Process

The early developments mentioned provided the key ideas of symbol and mechanism. From the earliest writings about these ideas—by such people as Turing, Von Neumann, Shannon, Wiener, McCulloch, McCarthy, Minsky, Newell, and Simon—it was apparent that what was incubating was not just another industrial technology but an intellectual tool that was directly relevant to unraveling the mystery of

intelligence (two of the earliest papers on this issue, published in the same year, are still worth reading: Turing, 1950, and Shannon, 1950).

As I have suggested, the formal universality of computation made it most attractive. Such universality implied that any process specifiable formally (in the sense, for example, required by a Turing machine) can be performed by a computer. More precisely, a computer can compute a formally equivalent function, which means that, in principle, it can be made to exhibit the same input-output behavior as that of a specified function. This "black box" equivalence is now considered a weak equivalence criterion. As we see below, in the case of cognitive psychology, explanatory adequacy depends on a stronger sense of equivalence, particularly on knowing the details of the process at a suitable level of abstraction. What, then, recommends computation as the appropriate vehicle for *that* task? To provide a framework for discussing this question, let us first look at computation from a more abstract point of view. That will help bring out further similarities in the relationship of computational devices and computational processes, on the one hand, and brains and cognitive processes, on the other. It is the failure to distinguish computation as a type of process from the particular physical form it takes in current computing machines that has prevented many people from taking computation as a literal account of mental process. If we understand computation at a fairly general level (as, in fact, it is understood in theoretical computer science), we can see that the idea that mental processing is computation is indeed a serious empirical hypothesis rather than a metaphor.

A computer is a physical object whose properties vary over time in accordance with the laws of physics. In principle, one might characterize a computer's dynamic behavior as a causally connected sequence of physical-state descriptions, with transitions subsumed under various physical laws. Because an unlimited number of physical properties and their combinations can be specified in the physical description, there is, in fact, an unlimited number of such sequences. By itself, however, the record of a particular sequence of physical states tells us nothing about the computational process going on in the machine, for two basic reasons.

(1) Very few of the physically discriminable properties of the machine are relevant to its computational function—for example, its color, temperature, and mass, as well as its very slow or very fast electrical fluctuations, variations in electrical properties above or below relevant ranges, variations in electrical properties that are within tolerance limits, and so on. In fact, *any* variations in physical properties of distinct components, other than those few properties to which other designated components are meant to react in *particular* specified ways, can be said

to be irrelevant to the machine's operation as a computer. Thus, in a given computer, only a minuscule subset of physically discriminable states are computationally discriminable. Of course, each of the infinitely many sequences of physical descriptions could correspond to some computational sequence. At least in principle, there is nothing to prevent such an interpretation. In other words, computational states are relativized to some convenient mapping, which I shall call the *instantiation function* (IF). By mapping from physical to computational states, such a function provides a way of interpreting a sequence of nomologically governed physical state changes as a *computation*, and therefore of viewing a physical object as a *computer*.

(2) A computational state of the machine corresponds to an equivalence class of physical states indistinguishable from the point of view of their function in the machine's abstract computational description. In typical computers, physical descriptions of these equivalence classes are extremely complex. To give a physical description of some arbitrary computational state of a computer involves an extremely long story covering myriad topographic and electrical properties and giving the ranges of parameters and combinations of parameter values that leave the state invariant.

Even for a particular computer the situation is much more complex than can be indicated here. Under the ordinary interpretation of "computational state," a computer executing a program written in, say, LISP is in a unique computational state whenever it begins with some given input and arrives at a certain point in the program. Of course, the class of physical states corresponding to *this* sense of computational state depends, among other things, on which LISP interpreter was used. Even a slight change in interpreter results in a radically different class of physical descriptions, corresponding to a particular LISP computational state. In fact, simply storing a program in a different sequence of memory registers of a stored-program computer radically alters the physical description corresponding to what would be considered the same computational state. This example shows that what I call the instantiation function is not fixed in a particular machine but can itself be altered, in this case under control of the interpreter, which translates from LISP statements to machine operations, or a loader program, which stores a program in specified storage locations. The point here is that there need be no uniform way of giving a physical description of the equivalence class of physical states of a machine that correspond to each of its computational states.

For a particular, finite computer, such a mapping from physical to computational states can, in principle, be found if we know the relevant physical parameters, since we can enumerate all relevantly discriminable

physical states; in general, however, because of the independence of structural and functional descriptions of states, this is not possible. This follows from the open-endedness of physical realizations of a given computational function. Not only can such computational sequences be realized in any digital computer ever made or that will ever be made, but they can be realized in devices operating in any imaginable media—mechanical (as in Charles Babbage's Analytical Engine), hydraulic, acoustical, optical, or organic—even a group of pigeons trained to peck as a Turing machine!

The reader will have recognized the similarity between the discussion above and the parallel arguments for the independence of cognitive processes from their underlying physical instantiation. Both computation and cognition are appropriately viewed as *rule-governed* processes. Computation is rule-governed because, as noted, regularities in the sequence of states a computation goes through are not expressible in terms of nomological laws, inasmuch as they depend on the instantiation function IF that specifies the equivalence class of physical states which count as a particular computational state. In addition, since IF is systematically alterable, as is also noted above, the regularities of the system are not lawlike, but rather have a rule-governed character. Similarly, cognitive processes are viewed as rule-governed, since the regularity in the sequences mental states go through is also nonnomological; that is, although the process is physical, its cognitive regularities cannot be explained by appeal to physical laws (see chapter 1). Also, the regularities can frequently be altered systematically by providing information (an important point discussed in chapter 5).

One of the most important similarities between cognition and computation, however, is suggested by the observation that the explanation of how a computation (in the usual sense of the term) proceeds must make reference to what is represented by the various intermediate states of the process just as the explanation of cognitive processes must make reference to the content of mental states.¹ In other words, the

1. Note that, in this sense, computer science is concerned with more than mere computation. It is concerned with certain functional properties of classes of devices that *can* be used for computing. Thus, much of computing theory is concerned with certain formal properties of symbol-manipulation systems, not with computation in the sense intended here. Nonetheless, the main interest in these results is that they bear on the question of what interpreted computations can be performed. Here, we have a close analogy with formal logic. Although, in proof theory, one investigates properties of various, formal, axiomatic systems, the motivating concern is with the notion of truth and with transformations that preserve truth value. It would thus be correct to view logic the same way as we view computation, namely, as being concerned with interpreted-symbol systems, the only difference being that logic is concerned primarily with the class of truth-preserving transformations, whereas computation is concerned with a wider range of classes of transformations—though, in the end, they remain semantically specified classes.

answer to the question "What computation is being performed?" requires discussion of semantically interpreted computational states. This central point can bear detailed discussion. Notice that the operation of a computer, viewed as a physical device, can be described in terms of the causal structure of the computer's physical properties. The states of a computer, when it is viewed as a physical device, are individuated in terms of the identity of physical descriptions; and therefore, its state transitions are connected by physical laws. By abstracting over these physical properties we can give a functional description of the device, which, for example, might be summarized as a finite-state transition diagram of the sort familiar in automata theory.

If, however, we wish to explain the *computation* being performed, or the regularities exhibited by a particular computer programmed to carry out a specific function over, say, numbers or sentences, we must refer to objects in the domain that are the intended interpretation, or *subject matter*, of the computations. (See the discussion in chapter 2 concerning the relation of explanations to the taxonomy under which we view the events being explained.) Thus, to explain why the machine prints the numeral "5" when it is provided with the expression "(PLUS 2 3)" (with the symbols given their usual interpretation), we must refer to the meaning of the symbols in both the expression and the printout. These meanings are the referents of the symbols in the domain of numbers. The explanation of why the particular numeral "5" is printed out follows from these semantic definitions; (that is, it prints out "5" because that symbol represents the number five, "PLUS" represents the addition operator applied to the referents of the other two symbols, and five is indeed the sum of the numbers two and three). In other words, from the definition of the symbols, or numerals, as representations of numbers, and from the definition of "PLUS," as representing a certain abstract mathematical operation (defined in terms of, say, the Peano axioms of number theory), it follows that some state of the machine, after reading the expression, will correspond to a state that represents the value of the function and (because of a further definition of the implicit printout function) cause the printing of the appropriate answer.

In addition to being dependent on the instantiation function IF, computational explanation requires another interpretation function, one that might be called the semantic function (SF), which maps articulated functional states onto some domain of intended interpretation. These semantic interpretations cannot, of course, be provided capriciously if the computation is to follow a given set of semantically interpreted rules. What is special about computers is that they are designed to make easy certain, particular, consistent interpretations. One way they

do this is to arrange for the initial state of the machine (that is, the hardware plus the operating system, or top-level language) to map in a straightforward way (via a simple IF) onto a variant of a Universal Machine. That way, the machine can be made to compute any function specifiable in its top-level language. Another way this is done is to have certain built-in regularities that allow substates of the machine to be easily interpreted in some generally useful domain, for example, numbers. The following example illustrates the provision of a simple SF in the design of symbol-manipulating mechanisms.

Syntax and Semantics in Computation

A Numerical Example

Suppose we have a certain IF that expresses a mapping from equivalence classes of physical states of a certain machine to symbolic expressions. The IF mapping might be defined by certain electronic circuits that cause the printing of strings of symbols on a printer, where the strings correspond to the states of certain parts of the machine called "memory registers." The states of the machine's memory registers could then be characterized in terms of certain symbolic expressions. Let us say that the expressions consist of the atomic symbols *o* and *x* arranged in strings of arbitrary length. In this example the states of the memory registers are designated by such expressions as *o*, *x*, *ox*, *xo*, *xx*, *oox*, *oxo*, *oxx*, *xoo*, *xox*, *xxo*, *xxx*, *xooo*, and so on; each expression designates some possible state of each machine's memory registers.² Let us further suppose that when a certain pattern (designated by the symbol " $\oplus$ ") occurs in a portion of the machine called its "instruction register," the machine's memory registers change states according to a certain, specifiable regularity. For example, when the portion of the machine called register 1 is in the state that maps onto the string *xox*, and register 2 is in the state that maps onto the string *xxo*, then register 3 changes its state from whatever it was to the state corresponding to the string *xoox*.

This regularity *might* be used to represent addition of numbers, providing we adopt an appropriate semantic function, SF, and that the

2. Note that, although I assume no bound on the length of the expressions, the points I will make hold as long as the length is sufficiently long that the expressions must be treated as though they were not atomic, something that is true in any useful computer. Therefore, there is a restriction on IF paralleling the restriction on how computational states are expressed, namely, that the expressions are instantiating *by means of* instantiating the elementary symbols *o* and *x* and the concatenation relation. We see below that another reason here is that the operation $\oplus$ (defined below) must also be defined over the individual symbols.

regularity meets certain requirements. Here, the required semantic function is easily defined; it happens to be the function that maps strings of o's and x's onto numbers, namely, the binary number system. Moreover, in defining the SF formally, we provide a way of stating the requirements the regularity must meet if it is to be consistently interpretable as addition of numbers. Before I define the SF, I must give a formal definition of the set of expressions consisting of x's and o's. Since I do not assume any bound on the number of states a register can take (hence, on the length of the strings of x's and o's), the definition of the strings must be given recursively as follows:

- (1) o is a string
- (2) x is a string
- (3) if T is a string, then so is To (string T followed by o)
- (4) if T is a string, then so is Tx (string T followed by "x")

A simpler way to express (1) to (4) is:

$$T \equiv o \vee x \vee To \vee Tx$$

(where $\equiv$ means "is defined to be" and $\vee$ means "or").

The semantic function can then be defined recursively as follows, using the definition of the strings provided above:

- (1) $SF[o] = 0$
(the semantic interpretation of "o" is the number zero)
- (2) $SF[x] = 1$
(the semantic interpretation of "x" is the number one)
- (3) $SF[To] = 2 \times SF[T]$
(The semantic interpretation of a string T followed by "o" is two times the semantic interpretation of T alone.)
- (4) $SF[Tx] = 2 \times SF[T] + 1$
(The semantic interpretation of a string T followed by x is twice the semantic interpretation of T alone plus 1.)

This is an example of a semantic function defined recursively on the structure of strings of symbols. It is analogous to Tarski's method of defining the semantics of sentences in some formal calculus in terms of their combinatorial properties. The mapping function is nontrivial; in fact, it defines the semantic interpretation of a *place-value* numeral notation.

For this semantic function to be useful, however, there must be regularities in the computer's state transitions that correspond to mathematical operations defined over the interpretations of the symbols in the domain intended. In other words, there must be state transitions

that *preserve the intended interpretation SF*. I have suggested such a regularity, one associated with the occurrence of the symbol $\oplus$ in the instruction register. For $\oplus$ to correspond to addition or, alternatively, for it to be consistently interpretable as addition, state transitions must preserve the semantic interpretation of the symbol strings under the mathematically defined operation of addition (defined, say, in terms of the Peano axioms). In other words, something like the following must be true:

If the computer is in the state characterized by the description:

- (1) register 1 "contains" (or IF maps it onto) string T'
- (2) register 2 "contains" (or IF maps it onto) string T''
- (3) instruction register "contains" (or IF maps it onto) the symbol, $\oplus$

then the computer goes into the state characterized by:

register 3 "contains" (or IF maps it onto) the string T , where

$$SF[T] = SF[T'] + SF[T''].$$

In other words, the mathematically defined sum of the semantic interpretations of the two register states must always correspond to the semantic interpretation of the state of the third register. Note that the interpretation is in the abstract domain of *numbers*, where such operations as additions are mathematically defined, whereas the symbols being interpreted (the domain of the SF function) are functional states, defined by IF as equivalence classes of physical states of the computer.

An interesting question remains: How is such an operation as $\oplus$ possible if the number of expressions is nearly unlimited? If the number were small one could imagine wiring up some sort of table-lookup (which is easy in a random-access memory). For a large or infinite number of expressions, however, that is not possible. What must be done instead is capitalize on the compositional nature of the semantic function and define simple rules that operate on individual x and o symbols.³ That, of course, is precisely what people do when they learn

3. Here, for example, is a set of eight production rules that will suffice. They map 5-tuples into 5-tuples. The 5-tuples are to be interpreted as: position of symbol; symbol in register 1 at that position; symbol in register 2 at that position; symbol in register 3 at that position; and carry symbol. The symbol "-" indicates that the symbol is irrelevant, and the n going to $n + 1$ is simply a way of indicating that after applying the rule to column n , one moves on to column $n + 1$ (right to left); no actual counting is required, only moving over one column.

$$\begin{array}{ll} (n,o,o,-,o) \rightarrow (n+1,-,o,o,o) & (n,o,o,-,x) \rightarrow (n+1,-,x,o,o) \\ (n,o,x,-,o) \rightarrow (n+1,-,x,o,o) & (n,o,x,-,x) \rightarrow (n+1,-,o,x,o) \\ (n,x,o,-,o) \rightarrow (n+1,-,o,x,o) & (n,x,o,-,x) \rightarrow (n+1,-,o,x,x) \\ (n,x,x,-,o) \rightarrow (n+1,-,o,x,x) & (n,x,x,-,x) \rightarrow (n+1,-,x,x,x) \end{array}$$

to add; they learn what to do with pairs of digits, then repeat the procedure indefinitely for larger numbers. Computers must do this, too, though there are some shortcuts for fixed-length expressions. That is why, as I have said, expressions are instantiated *by means of* instantiating the individual symbols and applying the concatenation operation. Biological instantiation of cognitive representations must also be realized by means of instantiation of component parts of the representation, so long as (a) there is an arbitrarily large number of representations, and (b) representations are transformed according to operations which, like the addition operation $\oplus$, respect semantical rules.

These ideas and distinctions, which arise in clear form in the case of conventional computers, are presented here because they help us keep straight what must be going on, since it becomes somewhat harder to think about these things when we take up the parallel case of mental processes in the next section. What we discover there is that we need all three distinct levels (physical, symbolic, semantic), just as we need them in the computer case. We need the syntactic or the symbolic level because we must preserve certain interpretations over mental operations, just as we must preserve the numerical interpretation given by SF in the example above, over the operation designated $\oplus$. This we can do only if we have a semantic function whose definition has access to the generative structure of the symbolic expressions (assuming that the set of thoughts is unbounded, just as the set of symbolic expressions was assumed to be unbounded in the example above). In the next section I will lead up to this by giving some simple examples of rules that must preserve their semantic interpretations.

Cognitive Rules and Syntax

In discussing the notion of a universal mechanism earlier, I spoke of interpretation, or *designation*, as being a key relationship into which symbols in a computer enter. Symbols can designate other symbols or primitive operations in the machine. To count as a computation rather than simply any functionally described physical system, however, it must contain symbols that are interpreted. In other words, the symbols must represent numbers, letters, or words, etc. (the slogan here is: “no computation without representation”). This is a different kind of designation because it is not wired into the machine (for which all symbol tokens are meaningless, except that certain ones can be made to cause the execution of primitive operations). The designation is provided by a person who takes the symbols to be about something—that is, the person gives the symbols a semantic interpretation. Of course, the task of remembering and communicating to others what interpretation of internal symbols a user has in mind is greatly simplified by labeling

keys with letters and numbers and wiring certain states to cause letters and numbers to be printed. Still, the interpretation typically is effected by the user, not the machine or the input-output terminal. (We have already considered the vexing question of whether this is a matter of necessity or whether machines can have "original" semantics.)

This quality of symbols and of computational states, whereby they can consistently be given a semantic interpretation, is not the only thing that makes useful computation possible; but it is one of the most important characteristics shared by computation and cognition. Any description of a cognitive process that characterizes the process as a rule-governed sequence of semantically interpreted states can be mapped directly onto a computational description of the process. Although computation may not be the only way to realize a semantically described process, it is the only one we know how to achieve in a physical system. Put another way, nobody has any idea how it might be possible, even in principle, to build a system whose behavior is characterized in terms of semantic rules without first describing the system in terms of operations on symbol structures.

Let us see how we might convert into a concise formal statement the sort of description of behavioral regularities we have been citing as putative examples of cognitive generalizations. The attempt to do this will help illustrate the nature of a computational account and at the same time suggest why a level of articulation of the representational states is necessary, hence, why a level of syntax or symbol structure is required to explain such regularities.

In chapter 2, I claimed that some regularities in behavior can be captured only in terms of a description that refers to the representational content of cognitive states. In the example given, I suggest regularities, which I now refer to as rule-governed, and which might roughly be conveyed by such descriptions as:

R1: If you *perceive* an event *E* as an instance of the category "emergency" (call this cognitive state S_1), then create the goal to get help (call this S_2), and

R2: If you have the *goal* of getting help, and you *know* that help can be obtained by telephone (call this combined state S_4), then create the goal of locating a telephone (resulting in state S_5).

These represent rules which hold among cognitive states. The first rule, R1, holds between S_1 , the state in which you perceive the event as an emergency, and S_2 , the state in which your goal is to obtain help. Rule R2 refers to mental state S_4 ,—which is a conjunction of two components, one being identical to S_2 —and the other corresponds to the belief that help can be obtained by using a telephone. Rule R2 connects S_4 with

state S_5 , which represents the state of having the goal of locating a telephone. Such an account can, in fact, be made precise if not by providing operational definition of terms, at least by specifying the formal relationship that hold among the terms. Indeed, the account can be simply represented in a computer.

Two points should be noted about these rules. First, in this explanation of the behavior, we describe R1 as holding between S_1 and S_2 by referring to what S_1 and S_2 represent, in terms of their semantic interpretation or their content. At this stage it might seem that any pair of states connected by a transition rule can be made to represent this regularity merely by interpreting the states appropriately. From such a perspective the system can be described by the following two state-transition rules:

R1: $S_1 \rightarrow S_2$

R2: $S_4 \rightarrow S_5$

On reflection, that cannot be the case—and this is the second point. The cognitive rule, as stated, relates semantic *contents* and is dependent on the presence of component parts of these contents (for example, on having the goal of obtaining help and holding the belief that help can be obtained by using a telephone to dial a certain number). Hence, when we express the rule in terms of state transitions, all rule-relevant aspects or components of the content must be distinguished as well. For instance, R2 must refer to the S_2 component of the S_4 state in a way that identifies it as the same representational content that appears in R1. Thus, R2 cannot be expressed simply as a rule between S_4 and S_5 ; the inclusion of two distinct parts of S_4 must be explicitly represented in R2, and one part must be identified as being identical to the consequent state of rule R1. Similarly, we will find that even S_1 and S_2 are not sufficiently fine decompositions; the fact that S_1 is a state that results from perception rather than from inference makes a difference to some further cognitive processing, as does the fact that S_2 is a goal state rather than a belief state. Therefore, such distinctions must be encoded by some functional feature of the state.

The moral here is that, to represent the system of rules holding among interpreted states, we cannot continue to refer only to sequences of distinct states, as I have been doing. We must refer to the states not as atomic entities but as *expressions* in which all cognitive terms and relations are mentioned explicitly. Cognitive processes thus must be represented as sequences of symbolic expressions. The terms of the expressions correspond to aspects of cognitive states to which cognitive rules are responsive. We thus conclude that, to account for behavioral regularities we must have terms which (a) provide distinctions among

cognitively distinct states, that is, states individuated in terms of their content; and (b) at the same time reveal the structure existing among distinct states, and show how various subsets of states share common features. For example, the term *goal* is necessary because goal states differ predictably from non-goal states, in that certain rules apply to certain states by virtue of the fact that they are goal states. This is true of such verbs as *see*, *believe*, *know*, and *hypothesize*.⁴

The situation here is similar to that which obtains in, say, phonology (a point first brought to everyone's attention by de Saussure, 1959). To account for the relationship among sounds when the sounds represent linguistic codes, it is necessary to analyze them into component parts called phonemes, each of which accounts for a potentially linguistic distinction. Phonological rules (which distinguish between well- and ill-formed linguistic sounds in a language and account for certain systematic relations among sounds, for example *contrite:contrition*, *ignite:ignition*) can then be stated over phonemic units. No such regularities can be captured if the rules must be stated over either unanalyzed whole utterances or sounds described in acoustical terms.

It is a general property of events and physical objects that if they are instances (token occurrences) of a *code* that designates something extrinsic to the physical system (that is, which is given a semantic interpretation), then, even though physical laws govern the properties of the *instances* (for example, their shape and succession if they are generated mechanically), the relationship among the events *as* codes must be stated in the form of rules that depend on what they are codes *for*. Anticipating an example used in chapter 7, I point out that patterns of Morse code follow rules of spelling because they are codes for letters. To express this, however, we must refer to what the codes represent: letters. Furthermore, the argument above is that this dependence of rules on interpretation means the codes themselves must explicitly

4. There is a question whether such verbs of "propositional attitude" as *believing*, *desiring*, *fearing*, and so on should be encoded as explicit symbols or whether they should be distinguished one from another in some nonsymbolic manner (for example, by storing their object clauses—the particular belief or desire—in different places). It seems that if such terms are symbolically encoded as in the examples above, they will behave differently from other symbols in the expression. For example, belief sentences do not allow existential generalization; from the fact that someone believes that "Bradford is a wise publisher," one may not infer that there exists someone who is a wise publisher (namely, Bradford), though, from "Harry believes Bradford is a wise publisher," one may conclude that Harry believes something, namely, that Bradford is. . . , etc., which shows there is something special about the verb *believe* itself that is not shared by other terms within its that-clause object. There are those who are much impressed by such facts. For the present, I think I will let the examples above stand as they are, since they provide convenient illustrations.

carry all relevant aspects of the interpretation as part of their intrinsic physical and functional *form*. This is exactly what is involved in the example given in the last section, where I describe an elementary computing system capable of going through state transitions that can be interpreted consistently as arithmetical operations. To characterize this constraint we mapped the substates of the system onto symbolic expressions. The expression-transforming operations were then given a finitary characterization so they could be interpreted, for instance, as adding numbers (as we do in our example). Here is the basic argument again, in a slightly more formal style. According to the computational theory of mind, two criteria must be met in providing an explanation of the behavior of an intentional or representational system.

(1) We must represent the rule-governed behavior of the system by giving formal rules holding among codes or among articulated cognitive states. In a formal system that is all the system has access to. It does not have the interpretations; only the theory provides that.

(2) As is argued above, though, if the system is representational, so that regularities in its behavior can be captured only by referring to the content of its representations, then the rules must have the property that those that apply to a particular code or state will appear to depend on what it is a code *for*, or, to cast it in the terms I used in speaking of the arithmetic example, they must “respect the semantic interpretation.”

These two criteria place restrictions on the form the code (or the description of the state) can take. In particular, a formal distinction must be made among the codes that corresponds to each rule-relevant distinction among the codes' interpretations: codes must preserve rule-relevant distinctions. For example, suppose $C_1, C_2, \dots, C_n$ are codes, and $I_1, I_2, \dots, I_n$ are their respective semantic interpretations. Suppose, further, that a set of interpretations $\{I_i\}$ differs from another disjoint set $\{I_j\}$ by virtue of the former having some semantic property or satisfying some predicate P (in the domain of interpretation) not satisfied by the latter. If there exists a rule whose applicability depends on P (for example, the rule applies to some code just in case the interpretation of that code satisfies P), then the system must be able to decide for any C_i whether $P(I_i)$ holds, that is, whether the predicate is true of the interpretation of the code in question. Because the system has no access to the semantic interpretation, whether $P(I_i)$ holds must be decidable by examining the formal structure of C_i alone. If the cardinality of the set of C_i is finite, one way the system might be capable of deciding whether $P(I_i)$ holds is to look up the particular C_i in a table that indexed all the codes according to which P applies to them (there can be only

a finite number of distinct P 's in that case). This trick cannot be used, however, if the set of C_i is unbounded.

If the set of rule-relevant semantic discriminations is arbitrarily expandable (that is, if it is, in principle, unbounded), the only way the set of codes can reflect all rule-relevant distinctions, as required by the above condition, is itself to be expandable, that is, constructive or productive. We cannot have an arbitrarily expandable set of codes unless a procedure exists for constructing the codes from a finite set of elements. This, in turn, means that computational states must be structured so they correspond to syntactically articulated expressions, rather than being mere atomic names. The C_i must have a structure that makes it possible to formally distinguish (by virtue of the presence or absence of more elementary components or symbols, or by virtue of specifiable properties of their arrangement) those codes C_i for which $P(I_i)$ holds from the codes C_j ($i \neq j$) for which $P(I_j)$ fails to hold. This is true for each predicate P that discriminates among rules in the system.

Even where the set of semantic discriminations is bounded (a thoroughly unreasonable assumption in human cognition, where clearly we cannot place a bound on the set of distinguishable thoughts)—hence, in principle, the codes could be atomic—the assumption that they are not expressions would, in general, be implausible on other grounds. In this case, the paradigmatic relations among the codes reflecting the structure among the things represented (as inferred, for example, from generalization gradients, confusion matrices, and so on) would have to be accounted for in other ways. For example, they would have to be explained as arising from distinct families of rules applied to different codes. What this would amount to is, we would get the effect of structured codes by applying collections of rules which, in effect, re-created relevant syntactic structures solely on the basis of the code's identity. This redundant proliferation of rules, needed to accomplish something so natural in a system that assumes the codes themselves to be structured, would have to be independently motivated.⁵

Such considerations as these also explain why programs cannot be described adequately in terms of state-transition diagrams, for example, those used in the description of finite-state automata, which, among

5. Such motivation does sometimes exist. Occasionally, there is evidence that a representation behaves as though it were not articulated into component parts, although, in other respects, it exhibits a multidimensional generalization or a similarity structure. An example is the "word superiority effect," where there is evidence that a whole word is easier to recognize than any of its constituent parts. In such cases we might be inclined to adopt the view that the code is unitary or atomic and that the generalization structure results from the application of sets of rules (for example, the atomic code for a lexical item may be used to access other properties of the word, such as its spelling).

other things, show that when a machine is in some state S_i and receives input x , it will go on to state S_j ; but if it receives input y , it will go into state S_k , and so on (see figure 1). As we have seen, the rule for adding numbers in a computer cannot be directly expressed in a finitary manner as a rule for producing a transition from state S_n to state S_{n+1} ; rather, it must be expressed as a rule or set of rules for transforming an expression of numerals into a new expression (see the preceding section). A *semantic* rule such as that for addition applies to states with a particular semantic interpretation, say, as certain numbers. Changes in the machine's state, however, are the result of physical causes; that is, they are caused by certain physical properties of the machine. Consequently, the way the machine must work to be correctly described as following a semantic rule is, on every occasion in which the rule is invoked there must be physical properties of the machine's state capable of serving as physical codes for that semantic interpretation. In other words, for each distinct, rule-relevant semantic property a corresponding, distinct, physical property must be associated with that state. Distinct, semantic properties must be preserved by *some* distinct, physical properties; in fact, they must be exactly the same physical properties that cause the machine to behave as it does on that occasion. Furthermore, such articulation of states into distinct properties must correspond to the articulation of the semantic rule in terms of symbolic expressions. In other words, articulation of the states must be made explicit if we are to express the rules governing the computation and at the same time show how, in principle, such rules might be realized in a physical system.

Another way to state the above is to say that if, in order to express some computational regularity in a finitary manner, we must refer to a structure of symbols, then the structure of the expressions must be

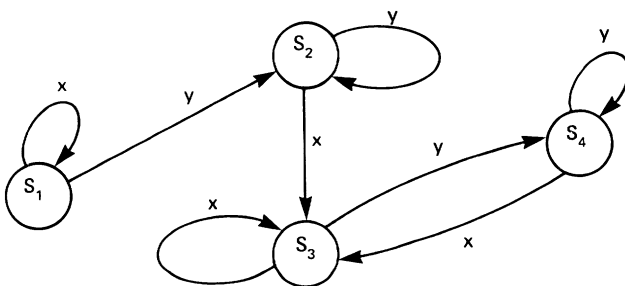


Figure 1. Diagram of a finite state automaton that reads strings of x 's and y 's. This automaton starts in state S_1 and stops in state S_4 just in case it reads a string containing one or more x 's that are not part of either an initial or final run of x 's.

reflected in the structure of the states of the system, as they were in our arithmetic example. Hence, it is not enough to describe the system as merely transforming state S_n into state S_{n+1} . That will not capture what is regular about the system's behavior in a finitary manner. Indeed, the machine itself must be designed so its physical operation will be governed not simply by the state of the entire device but by the substates of parts of the device exactly as described by the syntactic, or expression-transforming, description of the rules.

Thus: to capture the rule-governed quality of computation the process must be viewed in terms of operations on formal expressions rather than in terms of state transitions. Consideration of this view provides insight into why a computer (or a brain, for that matter) is more appropriately described as a Turing machine than as a finite-state automaton, though clearly it is finite in its resources. Notice that the finitude of the machine has a rather different status in the expression-transformation description than it does in the state-transition description. In the finite-state automaton case, the finiteness of the device is both intrinsic to that device and rigid, in the following sense: To increase the range of computation, one may increase the number of states. To do that in the finite-state automaton representation, however, one must, in effect, provide a new machine description, for example, a new finite-state diagram. In a "transformation on expressions" representation (or the more usual computer program representation), the device's finiteness is factorable from its computational process. To increase the range of computations or the range of expressions computed, one merely specifies an increase in the length of allowable expressions. The *procedure* and the *machine limits* are kept conveniently separate, as would seem desirable in view of the fact that altering the two is a fundamentally different kind of operation. In particular, changing the limits of either machine or brain can, in principle, be carried out indefinitely by providing more external storage (for example, magnetic tape or pencil and paper). (For more on this issue as it relates to the competence-performance distinction, see Pylyshyn, 1973a.)

A Note on the Notion of "Computing"

Above, I take a view of computation which, I believe, is the general view in computer science. I should point out that despite some 50 years of study (starting with Turing's famous paper on computability), there is still no consensus on just what are the essential elements of computing. Because I am concerned in this book with the computational view of mind, it is of some interest to try to understand what that commits us to. Clear cases of computing—in both practical computers and theoretical devices—differ in many ways. Even more significantly, there is

some feeling around that radically different kinds of “computers” may be possible that might fall outside the characterization I give here. Since some of this speculation has been gaining currency in cognitive science (where the problems are so difficult that people are continually desperate for a different technology), it may be worth a brief digression to consider just how chauvinistic and shortsighted is the view I have been presenting here.

It is my view that to understand what is essential about computing—or, at least, the aspect of it that is relevant to cognitive science—it is mandatory that we preserve a number of distinctions. In particular, we must distinguish between a machine and a program, and between a program and the “rules and representations” that the program uses. The second distinction is less important to the present discussion; it merely acknowledges that some symbols in the computer are semantically interpreted in some domain, whereas others are not, and that such semantically interpreted symbols alone are not enough to explain the actual behavior of the system (see chapter 2). We also need to know something about the control structure and about exactly how the rules and representations are turned into behavior by the architecture of the system—for example, how the operations are sequenced, if, in fact, they are carried out serially. In the sense I use the term, a *program* makes this distinction explicit, though such programming languages as PROLOG hide the control aspect from the user in order to keep the representations’ semantics “clean” and mathematically tractable. Although I do not want to go into this distinction here, I should point out that it is closely related to the distinction discussed earlier in this chapter between the two classes of symbols on the tape of a universal Turing machine—one representing the inputs to the function, the other designating which Turing machine is to be simulated.

In my view, the other distinction—that between the program and the machine—is absolutely fundamental.⁶ In this chapter, I try to make this point in a number of ways, but it deserves another try, since it is a distinction people frequently try to dismiss (cf. Anderson and Hinton, 1981).

The difference between an extremely complex device characterized merely as proceeding through distinguishable states (but not processing symbols) and what I call a “computer” is precisely the difference between a device viewed as a complex finite-state automaton and one

6. It does not matter here whether by *the machine* one means the device described in the manufacturer’s manual or what is sometimes called the “virtual machine,” consisting of the raw machine plus an interpreter for some higher-level programming language. That is merely a conceptual distinction, since the virtual machine is no less a real physical machine than the one delivered from the manufacturer; it just has a different initial state.

viewed as a variant of a Turing machine. I shall use the term *finite-state automaton* as a general way of talking about any device whose operation is described without reference to the application of rules to symbolic expressions. Thus, in this view, the “new connectionist” machines described by Anderson and Hinton (1981) are finite-state automata characterizations, as are various “network” or “planar circuit” characterizations of computers, such as those studied in complexity theory (for example, Pippenger and Fischer, 1979) or in VLSI design theory (Chazelle and Monier, 1983).

The difference between Turing machines and finite-state automata might seem to be merely that one is infinite and the other finite; but that isn't so, as I suggest a few paragraphs earlier. It is true that the class of functions that can be computed by these two machines depends on the unboundedness of the Turing-machine tape. As far as the relevance of the notion of Turing machine to cognition is concerned, however, the potentially infinite length of the Turing-machine tape serves to force a kind of qualitative organization on the process. Although certain cognitive capacities may be strictly finite, the fact that they apply over very large domains (for example, the number of sentences we can understand is very large) means they must be dealt with *generatively*. In mathematics, when the cardinality of a set is allowed to expand without bounds, for many purposes it becomes necessary to characterize the set *intensionally* in terms of some effective, finitary specification of its members or of conditions for membership in the set, since the elements of the set can, in principle, no longer be listed. Instead of simply referring to sets of infinite cardinality, mathematical methods typically consider some finitary procedure for, say, pairing elements of two sets.

Although the unboundedness of a Turing-machine tape does have consequences (such as undecidability results) whose relevance to cognitive science at present is not clear, it is the touchstone for the fundamental distinction that I also want to press, namely, the distinction between a strictly finite *mechanism* (the Turing machine's finite-state “control box”) and a finite but unbounded string of *symbols*. If it weren't for that distinction it would not be possible to have a universal Turing machine. The finite characterization of machines that such a distinction gives us is crucial. Turing machines are individuated by their finite part—that is what allows them to be effectively enumerated. Similarly, a finite part is required for proof theory (the axioms and rules of inference must be specified in a finitary manner). Frege (1980, p. 6) points out that the notion of an infinite number of primitive axioms is incongruous because “it conflicts with one of the requirements of reason, which must be able to embrace all first principles in a survey.”⁷

7. I am grateful to William Demopoulos for bringing this point to my attention.

As I have stated, it isn't the unboundedness per se that is important for our purpose, it is the form of organization the condition imparts. After all, one could extend without limit the number of states of a finite-state automaton; yet, without some finitary characterization of state-transition regularities, we would be right where we are in the case of an infinite-axiom logical system: Such a characterization would not allow us to understand (or, as Frege puts it, "survey") the function such a machine is computing. It is no accident that allowing the length of a Turing-machine tape to increase without bounds leads to a device that can be described in semantic terms, say, as computing some numerical function; whereas allowing the number of states of a finite-state device to increase without bounds does not.⁸ A device possessing an infinite number of atomic (nonarticulated) states cannot be given an effective semantic interpretation because the mapping from states to a semantic model cannot be specified in a finitary manner. For that to be possible, the set of states would have to be characterized intensionally or generatively, as is the case with the arithmetic example described above.

It is important to understand that what is at stake here is the nature of the organization captured in a certain description. Clearly, an ordinary von Neumann-style computer can correctly be described as a finite-state automaton. Such a computer can also be given a true description at the circuit level and at what Newell (1982) refers to as the "register transfer level." It is only when the computer is described as *operating upon symbols* (and, in fact, only when it is viewed as processing the particular symbols that are interpreted semantically) that we can explain its input-output behavior in semantic terms, for example, in terms of doing arithmetic, playing chess, carrying out inferences, or whatever else the device may be doing. Describing the computer at the symbol level is to make the crucial distinction I have been arguing must be made: separate the finite mechanism from the (to a first approximation) arbitrarily expandable symbol structures.

8. Again, it must be emphasized that what is at issue here is the way a machine is characterized. An FSA, with unbounded states, can be made to simulate a Turing machine in some fairly standard ways. In doing so, however, the network simulation of the Turing machine must explicitly preserve the distinction between the finite control box and the symbols on the tape. For example, in Chazelle and Monier (1983), different parts of the circuit simulate the tape and the control box. My point here is not that an FSA with unbounded states is any less powerful than a Turing machine, only that the characterization of the machine must observe the distinction between symbol and control mechanisms, in order for a device to be both universal and a computer in the sense in which I am using the term, namely, a device that computes a semantically interpretable function.

Here is another example that illustrates the point that what is at stake is how well the description reveals the underlying processing regularities. Suppose we take a Turing machine and "truncate" it, modifying it by providing the machine with a *finite* tape marked at each end with a special symbol. We then slightly modify the machine's control box, or "program"; that is, we compose this particular Turing machine with another, simple one so that whenever it reads the special symbol, it lights up a sign reading "tilt," then halts. In a certain sense the new machine is formally equivalent to some finite automaton; that is, it computes the same function. A description of the finite-state machine, however, loses all trace of the function computed by the original machine. This happens because, as I remark above, the source of finitary limitation is not factored out from the function computed in the finite-state-automaton characterization. True, the new truncated Turing machine computes a different function from the one computed by the original Turing machine; but it is equally true that there is *some* relation between the two. In fact, there is a rather close relation, but a finite-state characterization has no way of showing this.

Those who object to the conventional view of computation frequently have in mind the implausibility of the mind working like an IBM machine. I have much sympathy for this view, as I keep saying; that is why it is so important in cognitive science to discover the correct functional architecture of the mind. I would not be the least surprised to find that the mind's architecture is so different from that of a von Neumann machine that it might not be recognized as one of *our* computers. No doubt, the correct architecture will contain a great deal of parallel processing. Many people think that a substantial degree of parallel processing makes a fundamental difference in what is considered computing. The issue, however, is not whether the mind is a serial computer or a highly parallel one; it is whether the mind processes symbols, whether it has rules and representations. A highly parallel system can process symbols in at least two ways. One way is that the system may be parallel only in how it implements its primitive functions; that is, the functional architecture may be neurally implemented in a highly parallel way. That much, of course, is true of the von Neumann computer. Its random-access memory mechanism requires a great deal of simultaneous activity throughout the memory. The other way is that the primitive operations need not form a total ordering in time. For example, many rules might be applied at once, as in the production system designed by Forgy (1979). Since each rule is symbolic, there is no conflict with my sense of computing here. Even in an architecture as radically parallel as Hewitt's ACTOR scheme (Hewitt, 1977), there

is no conflict with the sense of computing I claim must go on in the mind, because each actor processes *semantically interpreted symbols*.

My point here is, so long as cognition (human or otherwise) involves such semantic regularities as inferences, and so long as we view cognition as computing in any sense, we must view it as *computing over symbols*. No connectionist device, however complex, will do, nor will any analog computer; but that is the topic of chapter 7.

To summarize what I have said about computing: A computational process is one whose behavior is viewed as depending on the representational or semantic content of its states. This occurs by virtue of there being another level of structure—variously called the “symbol level” or the syntax or logical form—which possesses the following two essential properties. One, the formal syntactic structure of particular occurrences (tokens) of symbolic expressions corresponds to real physical differences in the system, differences that affect the relevant features of the system’s behavior. Two, the formal symbol structures mirror all relevant semantic distinctions to which the system is supposed to respond and continue to do so when certain semantically interpreted rules are applied to them, transforming them into new symbol structures. For every apparent, functionally relevant semantic distinction there is a corresponding syntactic distinction. Thus any semantic feature that conceivably can affect behavior must be syntactically encoded at the level of a formal symbol structure. By this means we arrange for the system’s behavior to be describable as responding to the content of its representations—to what is being represented—in a manner perfectly compatible with materialism.

The Role of Computer Implementation

Our discussion so far concerns information-processing theories which are cast at the symbolic, or computational, level. We have not specifically addressed computer-implementation or -simulation models. What is the significance of the computer itself to this endeavor?

What the computer enables us to do, at the most abstract level, is exploit certain facilities (in the form of wired-in operations and compilers or interpreters for “high-level languages”), for

representing cognitive states in terms of symbolic expressions systematically related to their interpretation (that is, in a manner which preserves relevant distinctions);

representing rules responsive to these symbols (and hence to the interpretations of the states); and then

mapping this system onto causal physical laws so the entire apparatus can run, or produce state-to-state or expression-to-expression transitions that preserve the interpretation's coherence.

Not only is this an important tool for the construction and empirical exploration of computational models, but it also provides a way of understanding the possibility of a physical basis for mind, or at least of seeing that physics and mentalism are compatible. It makes it possible as well to see the sense in which such mentalistic notions as "cognitive process" can be literally true descriptions of intellectual activity occurring in the human organism, since we have no trouble seeing programs as literally—not metaphorically—true descriptions of the computational processes occurring in a computer.

The existence of digital computers, however, has an importance to cognitive science that goes beyond merely providing an existence proof that certain types of semantically described processes are compatible with materialism. Digital computers have dramatically altered the criteria by which the success of theories of cognition is judged. In addition to requiring that the theory capture generalizations about behavior, the notion of *sufficiency* now permeates theory construction in cognitive science. The term *sufficiency* refers to the theory's ability to explain how actual instances of behaviors are generated. Further, the explanation of the generation of such behaviors must be constructive, that is, effectively computable or realizable by a formal mechanism, for example, a Turing machine.

Such notions as "constructive" and "effectively computable" are central as well to a movement in mathematics called constructivism or intuitionism; the latter term, however, carries philosophical assumptions not necessarily shared by constructivism in general. Though quite different in their goals and assumptions, constructivism in cognitive science (in the sense of the adherence to a mechanistic sufficiency condition) and constructivism in mathematics share common assumptions concerning the conditions under which a claim is considered meaningful. Both cognitive science and mathematical constructivism require that functions referred to in the theory be effective or mechanistically realizable. In constructivist mathematics, for example, existence proofs are acceptable only if they provide a method for generating the objects whose existence is being proved; it is not enough to show that the assumption of the nonexistence of such objects leads to a contradiction. Proof by contradiction is not acceptable; demonstration that assuming $\neg P$ leads to a contradiction is not considered a valid method of proving P , since the law of excluded middle is not a logical axiom in constructivist mathematics.

Correspondingly, in the cognitive-science approach, to demonstrate that a certain method or a certain set of principles can account for a particular intellectual skill, it is necessary to show constructively (by exhibiting an effective procedure) that the method or set of principles can generate instances of that skill. In both constructivist mathematics and computational psychology, *construction* means to realize by a formal mechanism, where the notion of mechanism, in turn, rests on the notion of effective computation, or Turing-machine realizability, that is, the notion of mechanism discussed above in this chapter. In cognitive science, however, much stronger constraints than effectiveness are applied in assessing theories.

Any symbolic function implemented on a general-purpose digital computer is, by definition, effectively computable; yet mere implementability means little in psychological theory, for a variety of reasons to be explored in the next two chapters. As noted, one reason is that we are concerned with a much stricter sense of equivalence than input-output equivalence or behavioral mimicry; at the very least, we are concerned as well with realizability within various independently motivated constraints, including, of course, some sort of bounded time and memory complexity (whatever precisely that comes to). (This question is examined in greater detail in chapters 4 and 5.) Another reason for this formal sense of realizability being of limited concern is that the functions we are interested in constructing frequently are neither well defined nor purely symbolic; that is, they are initially specified in terms of physical inputs from the environment and the behavior of the organism. Thus the further requirement—that we be able to implement the process in terms of an actual, running program that exhibits tokens of the behaviors in question, under the appropriate circumstances—has far-reaching consequences.

One of the clearest advantages of expressing a cognitive-process model in the form of a computer program is, it provides a remarkable intellectual prosthetic for dealing with complexity and for exploring both the entailments of a large set of proposed principles and their interactions. In the case of human cognition, there is every reason to believe that the underlying causal mechanisms are unique with respect to both the quantity and the quality of their complexity. While it is true that theoretical understanding involves discovering the most general, abstract principles underlying apparent complexity, and that every object of study is infinitely complex until we have a theory that focuses our attention on its causally relevant features, it remains true that, relative to certain boundary conditions we impose on the project, theories can differ radically in the sheer number of ineliminable principles they must appeal to.

A major factor in determining the complexity of the theory is the requirement placed on it of accounting for certain pretheoretically specified phenomena—for example, how we understand language, remember our past, decide what to do next in playing chess, what to say in conversational communication, how to plan a trip, and so on. Although such specification is crude, and though only the successes of our evolving theories can dictate exactly how we should formulate our questions and puzzles, pretheoretical specification is not arbitrarily malleable; the phenomena have features we *insist* the theory address. In addition, the theory itself will have features which we insist it should have (being constructive, mechanistic, compatible with materialism). Even if we assume that no initial presumptions are inviolable, such a background of pretheoretical requirements can still determine the minimum complexity a theory can get away with and remain an account of the original phenomena we wish to explain. Relative to a strong form of constructivism, a theory must explain how an arbitrarily large set of beliefs, goals, and utilities can be brought to bear in generating behavior and that is likely to require a large theoretical apparatus.

My point about complexity is not merely that a lot is going on—for, clearly, just as much is going on in the physical universe—but that, unlike the case in physics, all the complexity may potentially be relevant to the constructive explanation of *any* aspect of behavior. This point is related to the “maximal-plasticity” property of symbolic systems already discussed. I suggest that, given our metatheoretical constraints, there may be a principled lower bound on the complexity of the constructive process. We already know from certain mathematical results in recursion theory that functions exist whose computational complexity is arbitrarily great in *any* machine (Blum, 1967) and that there are theorems whose proofs are likewise arbitrarily long.⁹ What we might well have in cognitive science is a case where the functions we have pretheoretically picked out (for example, those involved in intelligent action) do not admit of a simple, constructive realization, given the constraints on such realizations we require for independent reasons (which include more than that they be effective; see chapter 4).

Explanation, of course, is not the same as construction. In explanation, we need to factor apart various antecedents of behavior and provide general principles at various levels. Still, if the constructivist meth-

9. This follows from Church’s undecidability theorem. The result may be stated more precisely, as follows: For any recursive function f , there exists a theorem ϕ for which no proofs of length less than the absolute value of $f(\phi)$ exist. For, if there were, we could use this fact to construct an effective decision procedure for the existence of a proof of ϕ , which is contrary to Church’s theorem. I am indebted to Keewatin Dewdney for drawing this result to my attention.

odology is to be applied to anything resembling what we now call cognitive phenomena, then complexity may be endogenous to our project. It would be premature to draw sweeping conclusions from this possibility; surely it does not warrant the claim that human behavior is too complex to succumb to a constructivist analysis, as von Neumann (1966) and Shaw (1971) suggest. Yet that may be a reason why implementation of theories on real computers is more than a matter of convenience. The intrinsic complexity of the process, given our sufficiency requirement, may be more than could ever be accommodated without this prosthetic. There is an analogy here with the constructive proof of the four-color theorem in topology (see Appel and Haken, 1977): the only proof of this old conjecture that has ever been produced involves analysis of a large number of cases, and is so complex that, in all likelihood, it would not have been possible without the help of a computer. Similarly, it may be that the computer is indispensable to the realization of a constructivist theory of mind.

Ironically, one of the behaviorists' main criticisms of mentalism has long been that it is empty because it is all too easy to explain why people do things by appealing to their beliefs and desires: No problem; show me a pattern of behavior, and there is some obvious story based on the person's beliefs and desires that will explain it. If we now demand that the story be framed as an operational model—that is, as a computer program—this criticism no longer seems plausible. Not only isn't it easy to come up with a coherent cognitive story, it seems pretty nearly impossible to do so. Recent successes of artificial intelligence in building "expert systems" attests to both the complexity of the problem and how far one can go merely by building a lot of knowledge into some fairly conventional (by computer-science standards) systems.

Accepting the sufficiency condition on theories raises an entire, new set of issues. We are forced to understand the role of a new set of constraints on the theory-building enterprise, constraints that arise because the model must generate actual tokens of behavior. For example, the requirement that we bridge the gap between general principles and actual performance raises technical issues previously neglected in psychological theorizing concerning the notion of control.

The Control Issue

The commitment to construction of a model that actually generates token behaviors forces one to confront the problem of how, and under what conditions, internal representations and rules are pressed into service as actions are generated. These are questions that concern *control* of the process. Although a central topic in computer science, they have

almost never been raised in a cognitive psychology not constrained by computational sufficiency. Indeed, one of the main criticisms leveled against the early work of such cognitive psychologists as Tolman is that their theories deal only with an organism's representations, or "mental maps"; having established something about the representations, however, they cannot say how the representations lead to action: the theories left the organism "lost in thought."

There is much more to understanding control structures than knowing how operations are sequenced. We are so used to thinking of procedures as sequences of instructions which continue on a fixed course until some conditional branch operation detects a specified condition that alternative organizations do not readily spring to mind; yet that is merely one type of organization of control, one in which control is *passed* along a linear sequence from operation to operation. When one operation finishes, it passes control to the next operation in line. In computer science and artificial intelligence, however, there is much interest in very different control schemes, schemes that may change psychologists' thinking about the range of possibilities available for converting representations into action. In what follows I briefly survey some issues that arise when one considers the problem of controlling the way in which processes unfold in response to representations, rules, and the contingencies of the environment. My purpose is not to describe the range of control structures currently being studied in computer science, but merely to provide an intuitive sense of some of the distinctions in this field and suggest that cognitive science has much to learn from this area of development. Such considerations are not likely to be raised without a commitment to realization of the process model on a computer. Also, because control issues are a major area of study in computer science, progress in developing computational models of cognitive processes very likely will depend on technical ideas originating in that field—and especially in artificial intelligence.

Historically, a major breakthrough in understanding the nature of control was the articulation of the idea of feedback through the environment. With that understanding, a certain balance of control was restored between a device or an organism and its environment. Although only the device is credited with having a goal, responsibility for its behavior is shared. At times, when the environment is passive, the initiative appears to come primarily from the device; whereas, at other times, the environment appears to intervene, and the initiative seems to go in the opposite direction. This notion of the responsibility for initiation of different actions is fundamental to understanding control. In most computer programs the most common idea has been that of control moving from point to point, or from instruction to instruction,

in a largely predetermined way. This sequencing of instructions makes the notion of *flow* of control quite natural; branch instructions make it equally natural to think of *passing* or *sending* control to some other "place." When control passing is combined with a primitive message-passing facility—at minimum, a reminder of where the control came from, so it can be returned there later—*subroutines* become possible. And, because subroutines can be nested—that is, send control to still lower subroutines, and so on, with the assurance that control will eventually find its way back—the notion of a hierarchy of control emerges. Miller, Galanter, and Pribram (1960), who saw the psychological importance of the idea of hierarchical subroutines, called them test-operate-test-exit or TOTE units and suggested they be viewed as the basic theoretical unit of psychology, replacing the ubiquitous reflex arc. This idea has been highly influential in shaping psychologists' thinking about cognition.

There are several good reasons why a hierarchical system of control is so powerful (for a discussion, see Simon [1969]), one of the most important being that it results in "partial decomposability" or "modularity" of the total system. By keeping the interactions between routine and subroutine simple (in terms of when control is passed and the messages sent along with it), it becomes easier to think of each routine as a nearly independent subsystem, which makes the overall system easier to add to, modify, and understand. Moreover, each routine in the hierarchy can be viewed as defining some subgoal in an overall goal-directed system. Passing control to a subroutine amounts to activating or setting a subgoal; control is returned when the subgoal is achieved. So powerful is this idea that for years its shortcomings were largely overlooked. As early as 1962, however, Allen Newell (1962) pointed out some rigidity in this organization. So long as each subroutine is a narrow "specialist," Newell said, the minimal, rigidly prescribed communication between routine and subroutine works well. If it is a square-root extractor, you can just send it control and a number, and it will perform its function on that number independently of anything else in the data base. If, however, the subroutine is a less narrow specialist, it might help to be able to communicate each task in more flexible terms. Further, it might help to monitor the subroutine's progress along the way, to prevent it from consuming an unwarranted amount of time and resources (for example, memory) on some relatively minor task or on a task another process is in a position to determine is doomed to fail. Similarly, it would help for the subroutine to report its results more flexibly—especially if it could report "what went wrong" in cases of failure. How to convert these anthropomorphically stated desiderata into mechanical form, and how to do so without swamping the system

in a bureaucratic nightmare of control messages, is a major design concern in recent artificial intelligence programming languages (for a description of some of these languages, see Bobrow and Raphael, [1974]).

Because differences in control structure can have implications for psychological theory, and because these implications are not always obvious, I shall outline some general aspects of the problem. The aspects are stated in terms of the notion of *locus of responsibility* for such actions as transferring control and passing messages. I make two particular distinctions between (1) *sending* control (where the initiative lies with the old locus) and *capturing* control (where the initiative lies with the new locus), and (2) *directing* a message to a specific recipient and broadcasting it to all routines, or "modules," at once, that is, "to whom it may concern." Various control structures can be characterized on the basis of these distinctions. In the standard subroutine-hierarchy case, for example, control is always sent (by the routine that already has it), and a message (containing parameters and a return address) is directed specifically to the routine being given control. When the subgoal is achieved, control is sent back, along with a result message. In PLANNER-type languages (see Bobrow and Raphael, 1974), when a task is to be done, a message describing the goal is broadcast, then control is captured by a module designed to respond to that particular goal message. Next, control is sent to another location, one that depends on how that execution terminated (for example, whether it is successful, or on what result it produces). In a "production system," another control scheme that is becoming popular in certain areas of artificial intelligence and cognitive science, messages are also broadcast and control is captured. When the module (or "production") finishes, however, it again merely broadcasts a message; rarely is control sent to a specific locus. We will return to some implications of these choices; first, let us see how they show up in some simple examples and in the design of certain computer systems.

Remembering often consists of arranging the environment to trigger appropriate processes at appropriate times. People used to tie knots in their handkerchiefs or set up unusual situations to act as general "meta-reminders." Whereas mnemonic cues such as those used to improve one's memory are usually content-oriented access keys, memories often are evoked by less specific signals from the environment. Such signals are called "interrupts," because the initiative for capturing control originates outside the process currently active—in fact, outside the organism. The distinction between an interrupt and a test is relevant to understanding transducers, and will be discussed again in chapter 6. In addition, the distinction between data-directed, or event-directed, processing responsibility (characteristic of what are called "demon"

procedures, as well as of "if-added," "if-altered," or "if-examined" procedures evoked under various, special, recognized conditions) and process-directed control responsibility (characteristic of what are sometimes called "servant" or "if-needed" procedures) is an important, recurring theme in the study of control regimes.

Attempts to work out a balance between process-directed and data-directed procedure invocation is a major concern in the design of control structures for computational systems. Indeed, as the study of programming systems progresses, it has become apparent that the distinction between commands and other types of representation is more subtle than appears at first; the distinction rests on the role a particular symbol structure plays in the system. For instance, it is possible to have programming systems (for example, PROLOG) in which the notion of a command does not enter explicitly, in which everything takes the form of an assertion. In such cases it becomes apparent that the principal question is not whether something is a procedure or a datum but how information about control is represented, whether explicitly, so it is programmable, or implicitly (compare Kowalski, 1979). On the other hand, in some programming systems (for example, the ACTOR system of Hewitt, 1977), everything looks like a command or a procedure. In these systems control is completely decentralized, and the responsibility for initiating various procedures is distributed to the objects on which the procedures operate, instead of always coming from whatever procedure executed previously. People are just beginning to analyze in general terms these important issues in computer science. Clearly, the issues are relevant to an understanding of possible ways such knowledge systems as minds might be organized from the perspective of control.

As a final example of a novel control scheme, one that carries the idea of distributed control even further, I discuss a specific kind of system especially well suited for designing psychological models: the "production system" formalism developed by Newell (1973b). I mention it here briefly; my purpose is not to discuss the system's advantages and disadvantages in detail but merely to illustrate some benefits of thinking in terms of nonstandard control structures.

A production system has two main parts—a communication area, called the workspace, and a set of condition-action pairs, called productions. If the condition side of a production is satisfied by the current contents of the workspace, then that production is said to be evoked, and the action on its action side is carried out. The workspace resembles a public bulletin board, with the productions resembling simple minded bureaucrats who do little except look at the bulletin board. Each bureaucrat, however, is looking for a certain pattern of "notices" on the board. When it finds that pattern, the "bureaucrat" performs whatever

its action is—usually merely posting a notice of its own (except that some actions are transductions and perform input or output operations). In Newell's (1973b) version of production systems there is, strictly speaking, no explicit control-transfer operation (called "go-to" in conventional programming languages) or subroutine; that is, no bureaucrat ever gives any orders nor delegates authority, nor even sends messages to any other bureaucrat. All messages are broadcast (the contents of the workspace are visible to all productions), and control is always captured by a production whose conditions happen to be satisfied by the current workspace contents. The system is completely homogeneous, distributed, and modular.

Even on the basis of this brief description, we can see that production systems have attractive properties from a psychologist's point of view:

1. The system is responsive to a limited number of symbols at a time, which may be thought of as being in its focal attention. The system is completely "data-directed," in the sense that nothing happens unless some symbols in the workspace initiate processing by evoking a production.
2. Since symbols in the workspace can originate either from an environmental interrupt or from the action of some production, data-directed and environment-directed effects are combined, thus permitting a uniform treatment of what psychologists call stimulus-bound and stimulus-independent activity.
3. The primitive operation is one of *recognizing* that the symbols in the workspace satisfy some condition. Such recognitions can be made as elementary or as complex as desired; choosing them, however, amounts to selecting the primitive operations of the functional architecture, which, for psychologists, is at least partly an empirical issue.
4. Because production systems are completely data-directed, whenever they are used as models they must *explicitly* model the process's control structure. That is, since no hidden control apparatus exists, the flow of control must be handled by putting appropriate symbols *in the workspace* to evoke the relevant actions. These symbols then identify goals. A typical production system contains many goal-setting and -consummating productions—with the interesting consequence that, at any moment, the contents of the workspace can be thought of as containing everything that occupies the system's attention, including all active goals, as well as the debris of recent processing.
5. Production systems are highly modular; therefore, they can be extended uniformly without making widespread alterations to the existing

system. Another benefit of this modularity is that individual productions tend to be meaningful components of the entire system, a characteristic that may make it easier to give them psychological interpretations in terms of beliefs and goals.

6. The workspace is the dynamic, working memory of the system; thus it is assumed to be small, constituting one psychological hypothesis about resource constraints on the processor. For example, in order to attend to more aspects, it is necessary to trade off space for time. A natural approach is to assign a single symbol to designate an entire group of symbols, which can then be reconstructed from that single symbol whenever necessary. This approach fits well with psychological evidence for the mnemonic process called "chunking" (see Johnson, 1972), and Newell (1972) has used it in a model for Sternberg's high-speed-memory-scan results.

The production system control regime, as well as some other schemes sketched above, represent just a few of the ways in which the strictly hierarchical organization of control is being challenged by the notion of dispersed responsibility; but there are many more issues surrounding the study of control structures. What these studies represent is an attempt to understand alternative types of functional architecture for computing. In chapters 4 and 5 we see that what processes are realizable in a system, and with what complexity, depends crucially on the system's functional architecture. New ideas, such as parallel-processing architectures and networks that can create new interconnections or perform such operations as set intersection primitively (for example, Fahlman, 1979), are broadening our horizon of conceivable ways to realize processes in physical devices.

Many problems encountered in designing new architectures can be reduced to the three questions: How can we enable flexible, effective communication among different processes or modules? How do we ensure that all relevant information (and as little irrelevant information as possible) is brought to bear in making decisions or inferences? How can the making of decisions be withheld and released until the appropriate time? The second question (sometimes called the reference window problem; Marr and Nishihara, 1977) has received a great deal of research, leading to some proposals for organizing knowledge so its relevance to particular topics can be determined easily (for instance, "frames," "scripts," "symbol-mapping" schemes, "label-propagation" schemes, and so on; see, for example, Nudel, 1983). The third question is of special concern to such psychologists as Bransford and Johnson (1973), who have demonstrated experimentally that many inferences are carried out in advance of being required for a particular task (for

example, when utterances are heard). Occasionally, making decisions or executing procedures must be withheld until the appropriate context is available. Several proposals for dealing with such linguistic problems as referential opacity rely on the notion of withholding interpretation of parts of sentences pending the appropriate context. For instance, Davies and Isard's discussion (1972) of language comprehension places considerable emphasis on the importance of withholding evaluation of procedures that attempt to identify the referents of various parts of an utterance until the appropriate time. Thus there is growing recognition among investigators interested in the problems of cognitive psychology that a variety of questions related to control must play a more prominent role.

Often it is the necessity to provide a total running system that forces these issues to the fore. Without that requirement, people typically have focused on what are sometimes called "permissive" (or, in computer science, "nondeterministic") rules—for example, the rules of logic or of grammar—which specify the permissible relations among representations rather than on the issue of the conditions under which such rules are invoked. There is no denying that such permissive rules are fundamental. As I have argued in various contexts, it is essential that we know what they are, because that will tell us what the organism knows, which, in turn, determines the organism's behavior. Without the distinction between *performance* theory and what Chomsky calls *competence* theory, without what McCarthy and Hayes (1969) refer to as the distinction between the epistemological problem and the heuristic one, we could find ourselves simply mimicking the most frequent behavior rather than inferring the underlying mechanisms. Nonetheless, according to the computational view, understanding a process also requires having a theory of what makes the process unfold as it does on particular occasions—and that, in turn, requires that issues of control be addressed as well.

The real danger in following the sufficiency, or "performance," route is that it could lead to constructing ad hoc systems whose only virtue is that they can perform tasks similar to those performed by people, perhaps making the same sort of errors. (Various writers have cautioned about this hazard; see Marr, 1977; Pylyshyn, 1978a, 1979c, 1981.) Constructing such objects of mimicry, however, is not what psychology is about. Psychologists pursue the goal of explanation, which means that, although we pursue the constructivist program, we must make sure our use of computer models is principled. This, in turn, means that stringent constraints must be applied to the theory construction task to ensure that the principles are separated from the ad hoc tailoring of the systems to make them fit the data. To put this in more familiar

terms, we must make sure we specify the constants of the model in order to determine whether there are fewer degrees of freedom than there are data points.

In chapter 4 we consider the application of constraints to computational models, in order to take such models literally as embodying the same processes as those occurring in the mind.